
CAHIER DES CHARGES : LYZARD.AI

PROJET : SaaS de Génération Instantanée de Landing Pages par IA.

DURÉE ESTIMÉE : 6 à 8 Semaines.

1. VISION ET OBJECTIFS DU PROJET

1.1 Concept

Lyzard.ai est une plateforme "Text-to-Site". L'utilisateur saisit une description de son activité, et l'IA génère en temps réel une Landing Page professionnelle (code React/Tailwind), incluant le design, le contenu textuel et les visuels.

1.2 Objectifs Stratégiques

- **Zéro Code :** Rendre la création web accessible aux non-développeurs.
 - **Vitesse :** Livraison d'une page prête à l'emploi en moins de 30 secondes.
 - **Exportabilité :** Permettre aux utilisateurs de télécharger le code source (ZIP).
-

2. ARCHITECTURE TECHNIQUE (CLOUD STACK)

L'application repose sur une architecture moderne découplée :

Composant	Technologie	Rôle
Backend	Laravel 11	API REST, Orchestration de l'IA, Logique métier.
Frontend	React (Vite)	Interface interactive, Dashboard, Preview Engine.

IA Engine	Claude 3.5 Sonnet	Génération de code structurel et sémantique (JSON/Code).
Database	Supabase (PostgreSQL)	Stockage des utilisateurs, projets et versions de code.
Auth	Supabase Auth	Gestion des sessions et sécurité des comptes.
Cloud Storage	Supabase Storage	Hébergement des fichiers médias et exports ZIP.

3. SPÉCIFICATIONS FONCTIONNELLES (DÉTAILLÉES)

3.1 Authentification et Gestion de Compte

- **Login/Signup** : Via Email/Password ou Google (JWT géré par Supabase).
- **Système de Crédits** : Un quota de générations (ex: 3 gratuites, puis achat de crédits).

3.2 Le "Lyzard Builder" (L'interface de Chat)

- **Prompt Intelligent** : Champ de saisie supportant le multi-langue (Darija inclus).
- **Streaming de Code** : Affichage de l'avancement de la génération.
- **Live Preview (Iframe)** : Rendu immédiat du code généré dans un environnement isolé (sandbox).
- **Mode Itératif** : Possibilité de modifier une section spécifique en discutant avec l'IA.

3.3 Gestion des Projets

- **Bibliothèque** : Dashboard listant tous les projets créés par l'utilisateur.
 - **Auto-Save** : Chaque modification est sauvegardée en base de données.
 - **Export ZIP** : Génération d'un dossier contenant index.html, script.js et tailwind.config.
-

4. MODÉLISATION DES DONNÉES (SUPABASE SQL)

- **Users (Profiles)** : ID, Nom, Email, Solde de Crédits.
- **Projects** : ID, User_ID, Titre, Description, Thumbnail.
- **Code_Versions** : ID, Project_ID, HTML_Code, Prompt_Used, Version_Number (pour permettre le "Undo").

5. ORGANISATION ET RÉPARTITION DU TRAVAIL

Phase	Backend (Fayssal - Laravel)	Frontend (Binôme - React)
Sem 1-2	Setup Laravel/Supabase & API Claude.	Design UI Figma & Setup React/Tailwind.
Sem 3-4	Logic de génération & Sauvegarde SQL.	Interface Chat & Preview Engine (Iframe).
Sem 5-6	Logic d'Export ZIP & Gestion Crédits.	Dashboard Projets & Optimisation Mobile.
Sem 7-8	Tests API & Sécurisation.	Déploiement (Vercel) & Bug Fixing.

6. CONTRAINTES ET LIVRABLES

- **Sécurité** : Les clés API (Anthropic/Supabase) ne doivent jamais apparaître côté client (toujours passer par Laravel).
- **Performance** : Temps de réponse de l'IA optimisé via des "System Prompts" courts et précis.
- **Livrables** :
 1. Code source sur GitHub (Repo Backend + Repo Frontend).
 2. Application déployée et accessible via URL.

3. Documentation technique d'utilisation.
